

GICP GNSS + IMU Integration Bug Report

Three related defects in the localization pipeline for the AV-24 Cybertruck race car. Bug #1 sits in **gicp_localization** (DLIO_plusplus, branch *ucb-roar*); Bugs #2 and #3 sit in the upstream **race_common** (branch *stable*) and matter for what GICP would consume if it were re-pointed at the VKS-fused output, as the Bug #1 fix recommends. Bug #3 in particular gates the Bug #1 fix — switching GICP to `/vks/filtered_odom` without fixing Bug #3 first inherits a 1.57 m phantom translation whenever P1 wins the VKS source vote.

Report	GICP GNSS+IMU Integration
Date filed	2026-05-23 (updated 2026-05-24)
Author	Allen Yang (allenyang@berkeley.edu)
Repository	augcog/DLIO_plusplus (branch: <i>ucb-roar</i>)
Related repo	airacingtech/race_common (branch: <i>stable</i>)
Bugs	3 (one in DLIO_plusplus, two in race_common)
Severity	Bug #1: HIGH (cross-check and recovery silently wrong)
	Bug #2: MEDIUM (corrupts <code>/vks/imu</code> acceleration during yaw)
	Bug #3: HIGH (1.57 m phantom translation on P1 fallback)

Executive summary

The GICP localization node currently subscribes *directly* to the per-source `filtered_odom` topics published by `race_common`'s per-vendor interface nodes (*novatel_interface*, *vectornav_interface*, *pointonnav_interface*). Each vendor's `filtered_odom` is referenced at a **different physical point** on the car, because each vendor's interface applies a different amount of internal frame compensation. Switching between sources in GICP's GNSS prioritization logic therefore produces step changes of **~0.14 m to ~1.64 m** in the ground-truth pose that GICP cross-checks against, contaminating `gt_pos_err_m` and potentially triggering false GT-recovery snaps that drag the GICP pose by the same distance.

Separately, the natural fix — subscribing to VKS's fused `/vks/filtered_odom` (which is already lever-arm-corrected to the vehicle CG) — exposes GICP to two more defects. **Bug #2:** VKS's `ImuSource::apply_lever_arm` appears to use the wrong sign in the centripetal term, corrupting `/vks/imu.linear_acceleration` during yaw motion. **Bug #3:** P1's Atlas device outputs at `base_link` (confirmed by device-config inspection) but the FusionEngine ROS driver labels its messages with `header.frame_id = "gps_antenna_top"`. VKS's TF-based translation-to-CG then applies an offset that has nothing to do with where the data actually lives, producing a **~1.57 m** phantom translation whenever P1 wins the VKS source vote (~0.92 m of which is vertical, since the antenna sits on the roof while CG is at rear-axle height). Bug #3 must be fixed before the Bug #1 recommendation can be safely applied.

Bug #1: GICP per-source filtered_odom causes frame jumps on source switch

Severity

HIGH. Silently wrong cross-check, false-positive recovery snaps, no warning to operators. Symptom magnitude is large (up to ~1.6 m for AV-24).

Affected code

Repo: augcog/DLIO_plusplus (branch *ucb-roar*)

File: gicp_localization/cfg/localization.yaml, gicp_localization/launch/localization_with_tf.launch.py

Description

GICP's GNSS-direct ingestion path (and the legacy gt_odom path) subscribes to one or more of:

- /gps_na/filtered_odom — published by novatel_interface — reference point: **novatel_a** URDF link
- /gps_nav/filtered_odom — published by vectornav_interface — reference point: **vectornav** URDF link
- /gps_p1/filtered_odom — published by pointonnav_interface — reference point: **likely base_link** (configured on the Atlas device itself; see §Evidence)

After conversion from UTM/ENU to map, GICP treats each sample as being at base_frame (currently set to novatel_a). It does not apply a per-source lever-arm transform. The result: each source's pose lands in the buffer at a different physical point on the car.

Evidence (race_common configuration)

The interface nodes apply a per-vendor rotation but no lever-arm spatial correction. Comparing imu.rpy across the three vendor interface yamls reveals the asymmetry:

```
# novatel_interface.param.yaml (IAC novatel_a) imu: rpy: [0.0, 0.0, -1.5708] # -pi/2 yaw, rotate sensor axes to FLU
# vectornav_interface.param.yaml imu: rpy: [0.0, 0.0, -1.5708] # -pi/2 yaw, same as above
# pointonnav_interface.param.yaml (Atlas) imu: rpy: [0.0, 0.0, 0.0] # identity -- device already aligns internally
```

The pointonnav_interface_node code applies the (identity) rotation and passes header = msg->header straight through. The actual spatial reference of Atlas's output is set on the device (FusionEngine device.lever_arm / device.output_frame) and is **not** visible in race_common. The imu.rpy=[0,0,0] contrasted with -pi/2 on the other two is strong circumstantial evidence that Atlas is doing device-side body-frame alignment — the standard automotive Atlas deployment outputs at the configured vehicle reference (base_link or rear-axle center).

AV-24 URDF reference offsets

Source	URDF link	Position in base_link (m)	Distance
NovAtel	novatel_a	(1.636, -0.075, 0.101)	1.64 m
VectorNav	vectornav	(1.754, 0.000, 0.118)	1.76 m
Point-One-Nav	pointonnav (URDF)	(1.898, 0.027, 0.450)	1.95 m
P1 (likely actual)	base_link (device-configured)	(0.000, 0)	0 m

Impact: step jumps on source switch

Switch	Step in cross-check (m)	Likely consequence
NovAtel <-> VectorNav	~0.14 m	Inflates gt_pos_err_m baseline; may trip thresholds at corners
NovAtel <-> P1	~1.64 m	Trips every reasonable gt_pos_err_m threshold; recovery snap drags GICP
VectorNav <-> P1	~1.76 m	Same as above; recovery snap drags GICP by 1.76 m

When GICP's GNSS priority logic switches sources mid-run (e.g., NovAtel RTK degrades to RTK_FLOAT and VectorNav takes over, or P1 is added as a third source), the position in the gt_odom buffer jumps by the URDF lever arm between the two sensors. Downstream consequences:

- The cross-check gt_pos_err_m spikes by the switch magnitude. If the spike exceeds operator thresholds, the SCAN DEBUG line shows a phantom GICP divergence that didn't actually happen.
- If gt_recovery/enable=true (current default), the snap fires on the next non-accepted scan and drags GICP's pose to the new source's reference point — a 1.6 m teleport instead of a recovery. The vehicle's actual position has not moved.
- Geometric observer is reset; velocity estimate is corrupted until GICP re-converges and updateState corrects the state.

Reproduction

1. Run gicp_localization with gt_odom/enable=true and the GNSS-direct path consuming two or more vendors' filtered_odom.
2. Hold the vehicle stationary; record gt_pos_err_m baseline. Switch which source the prioritization picks (by toggling the source's RTK status, or by manually swapping the subscriber in the launch).
3. Observe gt_pos_err_m step of magnitude equal to the URDF distance between the two sensors.
4. With gt_recovery/enable=true, the snap fires on the next rejection and the published localized_odom jumps by the same amount.

Recommended fix

Stop consuming the per-source filtered_odom topics. Subscribe instead to race_common's /vks/filtered_odom, which VKS already produces by lever-arm-correcting every per-source sample to a single canonical reference point (target_frame: cg in the IAC config). For full consistency with the race stack, also change GICP's base_frame to cg so the reported pose lines up with what control and planning expect.

Target configuration:

```
# gicp_localization/cfg/localization.yaml localization/base_frame: cg # was: novatel_a
localization/imu_frame: novatel_a # unchanged (raw IMU still at novatel_a) #
gicp_localization/launch/localization_with_tf.launch.py # Re-point gt_odom (or the new
GNSS-direct entry point) to: gt_odom_topic := /vks/filtered_odom # imu_topic stays at
/gps_na/imu (raw, at novatel_a)
```

Trade-offs to be aware of:

- GICP loses per-source RTK quality visibility. VKS's voting is the policy you inherit (subscribers.filtered_odom.order: [0, 1], currently NovAtel primary, P1 fallback for IAC; VectorNav not enrolled). If GICP needs per-source decisioning, request that VectorNav be added to VKS, or implement per-source lever-arm correction in GICP using URDF TFs.
- A VKS failure becomes a GICP failure. Manageable since all other race-stack consumers depend on VKS too.
- Keep IMU subscription on /gps_na/imu rather than /vks/imu. See Bug #2 below for why.

Bug #2: VKS apply_lever_arm centripetal sign error

Severity

MEDIUM. Corrupts /vks/imu.linear_acceleration during yaw motion. Magnitude scales with yaw rate squared and the IMU-to-CG lever arm: $\sim 0.39 \text{ m/s}^2$ at 1 rad/s yaw on AV-24, $\sim 1.5\text{--}3.5 \text{ m/s}^2$ at race-corner yaw rates. Does not affect /vks/filtered_odom (pose-only) but does affect any consumer that ingests /vks/imu for acceleration-based reasoning.

Affected code

Repo: airacingtech/race_common (branch *stable*)

File: src/localization/vehicle_kinematic_state/src/sources/imu.cpp, lines 145–169 (function ImuSource::apply_lever_arm)

Description

VKS's ImuSource::apply_lever_arm converts an incoming IMU reading (specific force at the sensor) into the equivalent reading at target_frame (CG on AV-24) by adding a yaw-only centripetal correction. The code reads:

```
if (!tf_helper->lookup_transform( imu_msg.header.frame_id, target_frame_, to_cg_tf,
imu_msg.header.stamp)) { return false; } const double rx =
to_cg_tf.transform.translation.x; // position of cg in imu frame const double ry =
to_cg_tf.transform.translation.y; const double wz = imu_msg.angular_velocity.z; //
Centripetal acceleration: a_cg += wz^2 * [rx, ry] // (tangential term requires angular
acceleration, not available) corrected_msg.linear_acceleration.x += wz * wz * rx;
corrected_msg.linear_acceleration.y += wz * wz * ry;
```

The sign is inverted. Standard rigid-body kinematics gives $a_I = a_B + \omega \times (\omega \times r_{B \rightarrow I})$ where $r_{B \rightarrow I}$ is the position of the IMU expressed in the CG frame. Solving for a_B :

```
a_cg = a_imu - omega x (omega x r_cg_to_imu)
```

For $\omega = (0, 0, wz)$ and $r_{cg \rightarrow imu} = (R_x, R_y, 0)$:

```
omega x (omega x r) = (-wz^2 * R_x, -wz^2 * R_y, 0) a_cg = a_imu - (-wz^2 * R_x, -wz^2 *
R_y, 0) = a_imu + (wz^2 * R_x, wz^2 * R_y, 0)
```

So $a_{cg}.x$ should be incremented by $wz^2 \cdot R_x$, where R_x is the position of the **IMU in the CG frame** (positive when IMU is forward of CG). The code instead uses $rx = to_cg_tf.translation.x$, which is the position of **CG in the IMU frame** (negative when IMU is forward of CG). Substituting $rx = -R_x$ into the code:

```
Code: a_cg.x += wz^2 * (-R_x) = a_imu.x - wz^2 * R_x Correct: a_cg.x += wz^2 * (+R_x) =
a_imu.x + wz^2 * R_x
```

The code subtracts the centripetal contribution that it should add, with the result that the corrected output is wrong in both sign and magnitude.

Sanity check (synthetic stationary turntable)

Place the car stationary on a turntable centered at CG. IMU is forward of CG by 0.388 m (per AV-24 URDF: novatel_a in cg frame). Spin at $\omega = 1 \text{ rad/s}$ yaw, ignore gravity for clarity.

- True acceleration at CG: **0** (stationary)
- True centripetal at IMU: $-wz^2 \cdot 0.388 = \textbf{-0.388 m/s}^2$ in body x (pointing toward CG)
- Accelerometer at IMU reads (specific force): $\textbf{-0.388 m/s}^2$ in body x

- Correct apply_lever_arm output: $-0.388 + 0.388 = 0$ (matches truth)
- Actual VKS output: $-0.388 + (-0.388) = -0.776 \text{ m/s}^2$ (wrong sign, magnitude doubled)

Impact

Only /vks/imu is affected. /vks/filtered_odom carries the pose, not acceleration, and its lever-arm correction is pure translation (not subject to this sign issue). For DLIO_plusplus, this means the recommendation in Bug #1 (subscribe to /vks/filtered_odom) is safe; only consumers that take /vks/imu as input are affected.

DLIO_plusplus has a structurally similar code path in callbackImu (lin_accel_bl += ang_vel_bl.cross(ang_vel_bl.cross(t))) with the same suspected sign issue, but it is **dormant** because the AV-24 yaml sets base_frame == imu_frame == novatel_a, making the TF translation zero. If the recommendation in Bug #1 is applied and base_frame is changed to cg, the DLIO code path activates and the sign needs to be fixed in DLIO first (replace += with -=).

Recommended fix (race common)

```
// src/localization/vehicle_kinematic_state/src/sources/imu.cpp ~line 165 // Before: //
corrected_msg.linear_acceleration.x += wz * wz * rx; //
corrected_msg.linear_acceleration.y += wz * wz * ry; // After (sign corrected):
corrected_msg.linear_acceleration.x -= wz * wz * rx; corrected_msg.linear_acceleration.y
-= wz * wz * ry; // Equivalent and arguably clearer: // Use the IMU-in-CG position
(positive when IMU is forward of CG) instead // of CG-in-IMU. // const double Rx = -rx; //
const double Ry = -ry; // corrected_msg.linear_acceleration.x += wz * wz * Rx; //
corrected_msg.linear_acceleration.y += wz * wz * Ry;
```

Verification step

Before merging the sign flip, run the synthetic check above on a stationary vehicle with the wheels rotated (or on a turntable). The x-component of /vks/imu.linear_acceleration should converge to ~ 0 with the fix and to $\sim -2 \cdot \omega^2 \cdot |r|$ without it. If both signs give the wrong answer, the convention assumption about to_cg_tf.translation is different from standard tf2 (worth checking before committing).

Bug #3: P1 frame_id mismatch causes ~1.57 m phantom translation in VKS

Severity

HIGH. Silent, no warning, triggers exactly when ground truth is most needed (on NovAtel failure, VKS falls back to P1 and the bug activates). 1.57 m total error per sample (1.27 m horizontal + 0.92 m vertical). Currently affects /vks/filtered_odom directly — any consumer that subscribes inherits the bug. GICP is not yet affected (it consumes per-source filtered_odom per Bug #1) but the Bug #1 fix recommendation directly hands GICP this defect unless Bug #3 is fixed first.

Affected code

Repo: airacingtech/race_common (branch *stable*)

Files:

- src/launch/iac_launch/param/gps_param/atlas/atlas.param.yaml (sets frame_id: "gps_antenna_top")
- airacingtech/pointonenav_interface, src/pointonenav_interface_node.cpp (passes header through unchanged)
- src/localization/vehicle_kinematic_state/src/sources/odometry.cpp (the TF lookup that bites)

Description

P1's Atlas device is configured with a device.lever_arm that outputs INS pose directly at base_link (confirmed via P1 FusionEngine config inspection on the vehicle). This is the standard automotive Atlas deployment and is the reason pointonenav_interface's yaml has imu.rpy = [0, 0, 0] — the device has already done both the body-frame rotation and the IMU→base_link translation internally.

However, the FusionEngine ROS driver stamps every outgoing message with header.frame_id = "gps_antenna_top" (set in atlas.param.yaml). This is a label, not a transform — the driver does not move the data, it just tags it. The label is **wrong**: it claims the pose is at gps_antenna_top when it is actually at base_link.

The pointonenav_interface node passes the header through unchanged (odom_msg.header = msg->header at pointonenav_interface_node.cpp line ~257), so /gps_p1/filtered_odom inherits the mislabeled gps_antenna_top frame_id.

VKS then ingests the sample and calls tf_helper->lookup_transform(msg.child_frame_id, target_frame_, ...) to translate the position from the (claimed) source frame to cg. The TF tree resolves gps_antenna_top→cg as a real, non-trivial body offset, and VKS applies that offset to the position. But the position was already at base_link, so the applied translation is doubly wrong: (a) it includes the gps_antenna_top→cg geometry that should not have been applied, and (b) it omits the base_link→cg geometry that should have been.

Magnitude (computed from AV-24 URDF)

From av24.urdf:

```
gps_antenna_top in base_link = (1.27135, 0.0, 0.91994) m cg in base_link = (1.24800, 0.0, 0.07700) m VKS computes translation gps_antenna_top -> cg (in body coords): cg - gps_antenna_top = (-0.02335, 0.0, -0.84294) Correct translation (data is at base_link, should go base_link -> cg): cg - base_link = (1.24800, 0.0, 0.07700) Error vector (in body coords): (-0.02335, 0, -0.84294) - (1.24800, 0, 0.07700) = (-1.27135, 0.0, -0.91994) Error magnitude: 1.5693 m (3D norm) horizontal component: 1.2713 m (mostly -x, behind the vehicle) vertical component: 0.9199 m (down, since antenna is on the roof)
```

Note that the error vector in body coords equals the negative of `gps_antenna_top`'s position in `base_link` — the phantom translation places the vehicle as though it were at the antenna's location relative to where it should be.

Impact on GICP (if Bug #1 fix is applied without fixing Bug #3 first)

Switching GICP to `/vks/filtered_odom` per the Bug #1 recommendation inherits this defect with the following behavior:

- **While VKS uses NovAtel** (primary, source 0 in IAC config): no error. GICP cross-check and recovery work correctly.
- **The moment NovAtel degrades and VKS falls back to P1** (source 1): GT pose in the buffer jumps by ~1.57 m. `gt_pos_err_m` spikes; with `gt_recovery/enable=true` the snap fires on the next non-accepted scan and drags GICP's pose by 1.57 m. This is exactly the moment when GICP needs a reliable backup; the fallback path silently delivers a worse pose than the IMU dead-reckoning it was supposed to rescue.
- **When NovAtel recovers**: another 1.57 m jump in the opposite direction. Two-step diagnostic noise that looks like a localization glitch but is actually a frame-label bug.

Reproduction

1. Park the vehicle at a known surveyed location. Note the position downstream consumers report when VKS is sourcing from NovAtel.
2. Disable NovAtel (kill its driver process or remove its antenna so RTK drops to NONE). VKS falls back to P1.
3. Observe a step of ~1.57 m in `/vks/filtered_odom.pose.position` — the vehicle has not moved.
4. Re-enable NovAtel. Observe a step of ~1.57 m back to the truth.

Recommended fix

Three viable options, in increasing scope:

Option A (simplest): set the FusionEngine driver frame id to base link

```
# src/launch/iac_launch/param/gps_param/atlas/atlas.param.yaml # Before: # frame_id:
"gps_antenna_top" # After (matches what the device actually outputs): frame_id:
"base_link"
```

VKS's TF lookup becomes `base_link`→`cg`, which is the correct geometry. The same fix needs to be applied to `hawaii_gokart_launch`'s atlas configs if they have the same mismatch.

Option B: override the frame_id in pointonnav_interface

If multiple downstream consumers expect the legacy `gps_antenna_top` label for other reasons, override the `frame_id` when republishing `filtered_odom` in `pointonnav_interface_node` (at the line `odom_msg.header = msg->header` add `odom_msg.header.frame_id = "base_link"`).

Option C: per-source skip in VKS apply_lever_arm

Add a config flag per VKS source — `subscribers.filtered_odom.sources[i].in_body_frame: true` — that tells VKS to skip the TF lookup and treat the position as already at `base_link` / `target_frame`. More flexible but a larger code change and adds config to keep in sync with vendor configs.

Option A is recommended as the minimum-risk fix: it's a one-line config change that aligns the label with the actual data, requires no code changes anywhere, and benefits all downstream consumers (VKS plus anything else that ingests `/gps_p1/filtered_odom` or `/atlas/pose_filtered`).

Verification

Stationary test: park at a surveyed location, record the position from `/atlas/pose_filtered` and from `/vks/filtered_odom` while VKS is sourcing from P1 (disable NovAtel). Both should report the base_link position (~0.39 m behind cg in body x, at rear-axle height). Before the fix, `/vks/filtered_odom` will be off by ~1.57 m from truth; after the fix, both topics should agree to within stddev.

Combined recommendation and rollout

The three bugs are not independent. **Bug #3 must land before Bug #1's fix can be applied safely**, because the Bug #1 recommendation hands GICP a topic (/vks/filtered_odom) that currently inherits Bug #3 on every P1 fallback. Bug #2 is independent of the GICP fix path and only matters for consumers of /vks/imu acceleration.

Step 1 (race_common, stable): fix P1 frame_id mismatch (Bug #3)

- Change atlas.param.yaml frame_id from "gps_antenna_top" to "base_link" for the IAC car. Apply the same fix to any other vehicle launch (hawaii_gokart_launch, rosbag_localization_launch) that uses the same atlas configuration.
- Verify with the stationary test: park, disable NovAtel, confirm /vks/filtered_odom matches truth within stddev when P1 is the active source.

Step 2 (DLIO_plusplus, ucb-roar): switch GICP to /vks/filtered_odom (Bug #1)

- Re-point the GNSS-direct / gt_odom subscriber to /vks/filtered_odom.
- Change localization/base_frame to cg.
- Leave localization/imu_frame as novatel_a; leave the IMU subscription on /gps_na/imu.
- (If keeping the GNSS-direct architecture as future work) request that VectorNav be added to VKS's filtered_odom_order so the VKS layer can vote between all three sources symmetrically.
- **Do not apply this step until Step 1 has landed in race_common stable**, or every P1 fallback will inject a 1.57 m phantom snap into GICP.

Step 3 (DLIO_plusplus, ucb-roar): fix the dormant IMU lever-arm sign

Pre-emptively fix the sign in callbackImu:

```
// gicp_localization/src/localization.cc ~line 2790 // Before: // lin_accel_bl +=  
ang_vel_bl.cross(ang_vel_bl.cross(t)); // After: lin_accel_bl -=  
ang_vel_bl.cross(ang_vel_bl.cross(t));
```

This is dormant on the current yaml (t == 0 when base_frame == imu_frame) but activates once Step 1 changes base_frame to cg. Without the fix, the IMU prior to GICP is corrupted during cornering by the same magnitude as Bug #2 in VKS.

Step 4 (race_common, stable): fix VKS centripetal sign (Bug #2)

Fix the sign in ImuSource::apply_lever_arm as described in Bug #2. Validate with the turntable test before merging. Not required for GICP if GICP doesn't consume /vks/imu, but matters for any other race-stack consumer that does.

Step 5: verification

- Stationary test: confirm gt_pos_err_m stays flat (no step) when GNSS source switches mid-recording.
- Constant-radius circle test at constant yaw rate: confirm /vks/imu.linear_acceleration matches expected centripetal magnitude/sign for the known wz and radius.
- Full lap replay: compare localized_odom between the current setup and the post-fix setup. Expect divergence at corners and during RTK degradations — this is the bug being fixed, not a regression.

Open questions / caveats

- **P1 device-side output frame: CONFIRMED at base_link.** Verified by FusionEngine device-config inspection on the vehicle. This eliminates the earlier uncertainty and is what surfaces Bug #3 as a concrete defect rather than a hypothesis.
- **VKS centripetal sign (Bug #2) verification.** Two independent codebases (VKS and DLIO_plusplus) use the same sign convention. Worth double-checking the tf2 lookup_transform semantic before merging the flip — if the analysis is wrong about which direction to_cg_tf.translation points, the fix should be no-op rather than reversed. The stationary-turntable test catches either error.
- **VKS doesn't subscribe to VectorNav in the IAC config today.** If the GNSS-direct path is keeping VectorNav-as-fallback as a feature, either request a VKS config change or implement per-source lever-arm correction in GICP.

References

airacingtech/race_common (branch stable):

- src/launch/iac_launch/launch/gps_launch/oem7_net_a.launch.py
- src/launch/iac_launch/launch/gps_launch/vectornav.launch.py
- src/launch/iac_launch/launch/gps_launch/atlas.launch.py
- src/launch/iac_launch/param/gps_param/novatel_a/novatel_interface.param.yaml
- src/launch/iac_launch/param/gps_param/vectornav/vectornav_interface.param.yaml
- src/launch/iac_launch/param/gps_param/atlas/pointonnav_interface.param.yaml
- src/launch/iac_launch/param/gps_param/atlas/atlas.param.yaml
- src/localization/vehicle_kinematic_state/src/sources/imu.cpp
- src/localization/vehicle_kinematic_state/param/vehicle_kinematic_state_iac_car.param.yaml

airacingtech/pointonnav_interface (branch art-iron):

- src/pointonnav_interface_node.cpp

augcog/DLIO_plusplus (branch ucb-roar):

- gicp_localization/src/localization.cc (callbackImu ~line 2780)
- gicp_localization/cfg/localization.yaml
- av24.urdf (joints novatel_a_joint, vectornav_joint, pointonnav_joint, cg_joint)